

Phase 1 · Step 1.1 — 점유맵(Occupancy Map)

Routing3D 플랜트 배관 3D 직교 라우팅 · 단위 mm · 기본 셀 50mm · 모듈 occupancy.py

1. 개요

플랜트 공간을 일정 크기(기본 50mm)의 정육면체 셀로 나눈 3차원 격자로 표현하고, 각 셀이 '장애물로 점유되었는지'를 True/False로 관리한다. 이후 직교 A* 탐색이 이 점유맵에 '이 셀로 지나갈 수 있는가?'를 질의한다.

등간격 복셀은 고해상도/대영역에서 셀 수가 폭발한다. 그래서 같은 질의 인터페이스(OccupancyMap 추상 클래스) 뒤에 세 가지 저장 방식을 두고 상황에 맞게 고른다. A* 등 사용자 코드는 인터페이스(is_blocked / in_bounds / bounds / to_world / to_cell)에만 의존하므로 백엔드를 자유롭게 교체할 수 있다.

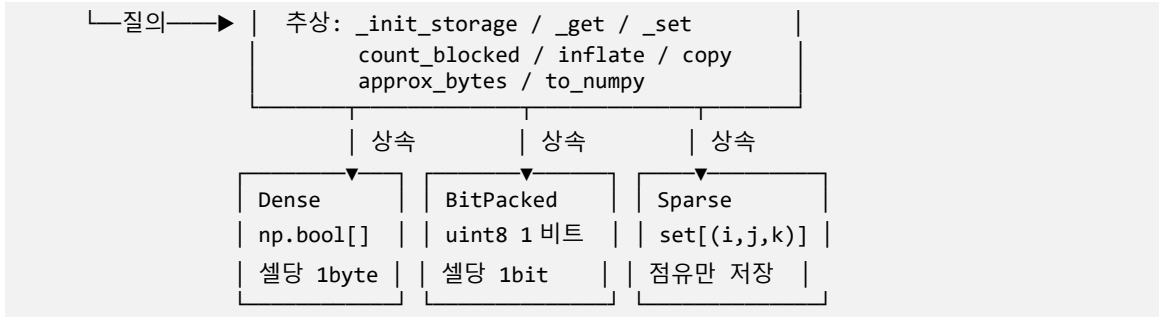
2. 단위·좌표 규약

- 모든 월드 좌표·치수의 단위는 밀리미터(mm).
- 셀 인덱스는 정수 3-튜플 (i, j, k). 셀 (0,0,0)은 origin에서 시작.
- 셀 중심 월드좌표 = $\text{origin} + (\text{cell} + 0.5) \times \text{cell_mm}$.
- $\text{shape} = (\text{nx}, \text{ny}, \text{nz})$ 는 각 축의 셀 개수.
- 격자 범위 밖의 셀은 항상 '점유(True)'로 간주 → A*가 격자 밖으로 못 나간다.

3. 전체 구조 흐름도

추상 베이스가 좌표변환·복셀화 등 '저장 방식과 무관한' 로직을 공통 구현하고, 실제 점유 비트의 저장/조회만 백엔드별로 구현한다.

사용자 코드	OccupancyMap (추상 베이스, ABC)
(A*, 비용함수, 시각화)	공통: in_bounds / bounds / world_bounds
	to_world / to_cell / is_blocked
	add_box / add_boxes / block_cell



4. 백엔드 비교

백엔드	셀당 메모리	질의 속도	적합 상황	실측(25.6% 점유, 50mm)
DenseOccupancyMap	1 byte	O(1) 최속	작은 ROI, 최속 질의(기본값)	625 KB
BitPackedOccupancyMap	1 bit	약간 느림	같은 메모리로 ~8 배 큰 ROI	78 KB
SparseOccupancyMap	set 엔트리	해시 조회	점유가 희박할 때만	23.6 MB

주의: 바닥·기둥처럼 점유가 뽕뽕하면 Sparse 는 set 오버헤드 때문에 오히려 Dense 의 ~37 배까지 커진다. 균일한 큰 덩어리의 진짜 압축(옥트리/VDB)은 433m 스케일이 목표인 Phase 3(OpenVDB)에서 도입한다.

5. 핵심 알고리즘

5.1 좌표 변환

- $\text{to_world}(\text{cell}) = \text{origin} + (\text{cell} + 0.5) \times \text{cell_mm} \rightarrow \text{셀 중심 월드좌표(mm)}$.
- $\text{to_cell}(\text{world}) = \text{floor}((\text{world} - \text{origin}) / \text{cell_mm}) \rightarrow \text{포함 셀 인덱스}$.

5.2 add_box — AABB 복셀화

- 1) lo/hi 월드좌표 $\rightarrow$ 셀 범위 (시작 = floor, 끝 = ceil, 끝은 제외 경계)
- 2) 격자 $[\text{lo}, \text{hi})$ 로 클리핑
- 3) 비어 있으면 0 반환, 아니면 `_fill_box` 로 채우고 '신규' 점유 셀 수 반환
 - Dense: NumPy 슬라이스 대입으로 가속 (`_fill_box` 오버라이드)
 - Sparse/BitPacked: 셀 단위 3 중 루프(기본 구현)

5.3 inflate — 장애물 팽창(하드 클리어런스)

장애물을 radius_cells 만큼 부풀린 같은 백엔드의 새 점유맵을 만든다. 1 단계 팽창은 _dilate_once(이웃 방향으로 OR-시프트)로 수행하며 radius_cells 번 반복한다. connectivity 6 = 맨해튼 볼(면 인접), 26 = 체비셰프 볼(면+모서리+꼭짓점).

```
_shift(grid, di,dj,dk) : 그리드를 (di,dj,dk)만큼 평행이동(범위 밖은 0)
_dilate_once(grid, off): result = grid; for o in off: result |= shift(grid,o)
inflate(r)             : for _ in range(r): grid = _dilate_once(grid, off)
```

6. 자료구조 / 클래스

이름	종류	설명
AABB	frozen dataclass	장애물 직육면체. lo<hi 검증. 필드 lo/hi = (x,y,z) mm.
OccupancyMap	ABC	질의 인터페이스 + 공통 로직(좌표/복셀화). 백엔드 공통 계약.
DenseOccupancyMap	구현	np.bool 3D 배열(grid). 가장 단순·최속.
BitPackedOccupancyMap	구현	uint8 배열(packed), z 8 셀=1byte, little 비트순서.
SparseOccupancyMap	구현	점유 셀 좌표 set(blocked).

7. 주요 함수

함수 / 메서드	역할
__init__(shape, origin, cell_mm)	shape/origin/cell_mm 검증·저장 후 백엔드 초기화.
from_world_bounds(lo, hi, cell_mm)	월드 범위를 덮는 빈 맵 생성(셀 단위 올림).
in_bounds(cell)	셀이 격자 범위 안인지.
is_blocked(cell)	점유 또는 격자 밖이면 True (A* 질의 핵심).
block_cell(cell)	단일 셀 점유 표시(격자 밖 무시).
add_box(box) / add_boxes(boxes)	AABB(들) 복셀화. 신규 점유 셀 수 반환.
to_world(cell) / to_cell(world)	셀 ↔ 월드좌표(mm) 변환.
world_bounds() / bounds()	격자가 덮는 월드 AABB / 셀 인덱스 범위.
count_blocked()	점유 셀 총 개수(백엔드별 구현).
inflate(radius, connectivity)	팽창된 새 맵 반환(백엔드별 구현).
to_numpy()	(nx,ny,nz) bool 배열로 펼침(시각화/내보내기/거리변환).
copy()	독립 복사본(다중 배관 순차 라우팅에서 원본

	보존용).
approx_bytes()	저장소 추정 바이트(백엔드 메모리 비교용).

8. 주요 변수 / 상수

이름	타입	설명
shape	(nx,ny,nz) int	각 축 셀 개수.
origin	np.float64[3]	격자 원점 월드좌표(mm).
cell_mm	float	셀 한 변 길이(mm). 기본 50, 설정 가능.
Cell	타입별칭	셀 인덱스 (i,j,k) 정수 3-튜플.
NEIGHBORS_6	tuple[Cell]	면 인접 6 방향($\pm x, \pm y, \pm z$). 직교 이동/6-연결 팽창.
NEIGHBORS_26	tuple[Cell]	면+모서리+꼭짓점 26 방향. 26-연결 팽창.
grid / packed / blocked	백엔드 저장소	Dense/BitPacked/Sparse 의 점유 저장 필드.

9. 실행 명령어

```
# 단위 테스트 (python_experiments/ 에서)
..\venv\Scripts\python.exe -m pytest tests/test_occupancy.py -v

# 코드에서 사용
from routing3d_py import DenseOccupancyMap, BitPackedOccupancyMap, AABB
occ = DenseOccupancyMap((40, 40, 40), origin=(0,0,0), cell_mm=50)
occ.add_box(AABB((100,100,100), (900,900,200))) # 신규 점유 셀 수 반환
occ.is_blocked((2, 2, 2)) # A* 질의
```